Q1) Create a SQL view named TopRatedRestaurants that selects the restaurant name, average rating, and total number of reviews from a table of Zomato-style restaurant reviews, showing only restaurants with an average rating above 4.0.

```
CREATE VIEW
TopRatedRestaurants AS
SELECT
    restaurant_name,
    AVG(rating) AS
average_rating,
    COUNT(*) AS
total_reviews
FROM RestaurantReviews
GROUP BY restaurant_name
HAVING AVG(rating) > 4.0;

SELECT * FROM
TopRatedRestaurants;
```

Q2) Update the TopRatedRestaurants view to also include the city column from the original restaurants table by joining the relevant tables.

Hint: Use an INNER JOIN to combine data from both tables in your view definition.

```
CREATE OR REPLACE VIEW
TopRatedRestaurants AS
SELECT
    rr.restaurant_name,
    r.city,
    AVG(rr.rating) AS
average_rating,
    COUNT(*) AS
total_reviews
FROM RestaurantReviews
rr
INNER JOIN Restaurants r
    ON
rr.restaurant_name =
r.restaurant_name
GROUP BY
rr.restaurant_name,
r.city
HAVING AVG(rr.rating) >
4.0;

SELECT * FROM
TopRatedRestaurants;
```

Q3) Try to update the average rating column directly through the TopRatedRestaurants view and observe what error or limitation occurs. Write down the exact error message and explain why this happens based on SQL view limitations.

```
UPDATE
TopRatedRestaurants
SET average_rating = 4.8
WHERE restaurant_name =
'Pizza Palace';
```

Q4) Create a view called DailyOrderSummary that shows, for each date, the total number of food orders and the total revenue from a Swiggy-style orders table. Ensure the view only includes dates from the last 30 days.

Constraint: Use WHERE and GROUP BY clauses in your view definition.

```
CREATE VIEW
DailyOrderSummary AS
SELECT
    DATE(order_date) AS
order_day,
    COUNT(*) AS
total_orders,
    SUM(total_amount) AS
total_revenue
FROM SwiggyOrders
WHERE order_date >=
CURDATE() - INTERVAL 30
DAY
GROUP BY
DATE(order_date);

SELECT * FROM
DailyOrderSummary
ORDER BY order_day DESC;
```

Q5) List 3 good practices you should follow when creating SQL views for analytics dashboards, and for each, give a one-line example related to a Flipkart sales reporting scenario.

1) Use meaningful column names

```sql
CREATE VIEW FlipkartSales
AS
SELECT product_name,
       SUM(sales_amount)
AS total_sales
FROM FlipkartOrders
GROUP BY product_name;
```

2) Filter unnecessary data

```sql
CREATE VIEW
CompletedFlipkartSales AS
SELECT order_id,
product_name,
sales_amount
FROM FlipkartOrders
WHERE order_status =
'Completed';
```

3) Aggregate data correctly

```sql
CREATE VIEW
FlipkartCategorySales AS
SELECT category,
       SUM(sales_amount)
AS total_revenue
FROM FlipkartOrders
GROUP BY category;
```